KallisteSoft Shorts: complete upload script (tools/daily_shorts.py)
Public source: https://github.com/kallistesoft/critter-logic/blob/main/tools/daily_shorts.py
Only API call: youtube.videos.insert in upload(); OAuth scope youtube.upload in youtube().

```
"""Daily YouTube Shorts: yesterday's daily puzzle, played on the emulator and cut into a 9:16
"Can you solve it?" clip.

    python3 tools/daily_shorts.py render [YYYY-MM-DD] [--days 7] [--lang en]   # record + edit
    python3 tools/daily_shorts.py login                                          # once: sign in
to the channel
    python3 tools/daily_shorts.py upload [--dry-run]                             # schedule on
YouTube

Each day gets one game (Nests, Pairs, Trail in turn) and the daily of the day before, so a Short
never shows the
solution of a puzzle someone can still play: it goes public at 17:00 New York time, when that
daily is over in every
time zone. Until the game is live on Google Play, the end card and description say "coming
soon".

render: the promo build (tools/promo_main.dart, see store/README.md) on an Android emulator,
staged on that daily with
no ranking at all (boards=false: the stand-in's player counts are made up). The puzzle is played
with its real solution
from assets/dailies. The edit (about 18 s, 1080 x 1920, 30 fps, H.264 + AAC) opens on the empty
board with the rules
and "pause and try it", plays the solve, holds the solved board and ends on a "Free on Google
Play" card, over the groove of
the promo music (tools/make_promo_music.py) and the game's solve sound. Writes
build/shorts/<date>-<lang>.mp4 and
<date>-<lang>.json (title, description, tags).

upload: every rendered Short not yet in store/shorts/uploads.json goes to the KallisteSoft
channel as private with a
publish time (publishAt = its date at 17:00 New York time), through the YouTube Data API. One-
time setup: see store/shorts/README.md.

Set PROMO_SERIAL to the emulator to use (default emulator-5554, never a phone).
"""
import argparse
import datetime as dt
import json
import os
import re
import subprocess
import sys
import time
import wave

from zoneinfo import ZoneInfo

import numpy as np
from PIL import Image, ImageDraw, ImageFilter, ImageFont

sys.path.insert(0, os.path.dirname(__file__))
from promo_device import (NESTS_BOX, PAIRS_BOX, PLAY, ROOT, daily, drag, grid, launch,
nests_cells,  # noqa: E402
                          pairs_taps, rec_start, rec_stop, setup, taps, teardown,
trail_drag_points)
from record_promo_clips import diffs  # noqa: E402

FFMPEG = os.environ.get('FFMPEG', 'ffmpeg')
OUT = os.path.join(ROOT, 'build', 'shorts')
LEDGER = os.path.join(ROOT, 'store', 'shorts', 'uploads.json')
```

```
KEYS = os.path.expanduser('~/Keys/kallistesoft')
CLIENT_SECRET = os.path.join(KEYS, 'youtube-client-secret.json')
TOKEN = os.path.join(KEYS, 'youtube-shorts-token.json')
PLAY_URL = 'https://play.google.com/store/apps/details?id=ca.kallistesoft.critterlogic'
GAMES = ['nests', 'pairs', 'trail']
# Every day at 17:00 New York time (14:00 Los Angeles, 16:00 Mexico City): Shorts posted shortly
before the evening
# peak of North American viewers do best. Always after 12:00 UTC, so yesterday's daily is over
everywhere.
PUBLISH_TZ, PUBLISH_HOUR = 'America/New_York', 17

W, H, FPS = 1080, 1920, 30
# The part of the 1080 x 2424 recording shown: the top bar with the timer down to just under the
board.
CROP = (0, 520, 1080, 1860)
FONT = os.path.join(ROOT, 'assets', 'fonts', 'Fredoka-Bold.ttf')
FONT_SEMI = os.path.join(ROOT, 'assets', 'fonts', 'Fredoka-SemiBold.ttf')
GOLD = (255, 216, 74)
INK = (42, 24, 64)
HOOK, HOLD, END = 3.0, 2.2, 2.6  # seconds: empty board, solved board, end card
MAX_PLAY = 9.0

TEXT = {
    'en': {
        'hook': 'Can you|solve it?', 'pause': 'Pause & try it', 'solved': 'Solved!',
        'nests': 'Nests', 'pairs': 'Pairs', 'trail': 'Trail',
        'nests_rules': 'One critter per row, column and zone. They never touch!',
        'pairs_rules': 'As many suns as moons in each line, never 3 alike in a row.',
        'trail_rules': 'One trail through every square, stones in order.',
        'free': 'Free on Google Play', 'soon': 'Coming soon to Google Play', 'end_sub': 'New
puzzles every day',
        'title': 'Can you solve this {game} puzzle? Daily #{n} #shorts',
        'desc': ('Daily #{n}: {rules}\nPause and try it before the solve!\n\n'
                 'Critter Logic: 3 new logic puzzles every day (Nests, Pairs, Trail), cute
critters to collect. '
                 '{free}:\n{url}\n\n#puzzle #logicpuzzle #brainteaser #dailypuzzle
#critterlogic'),
        'tags': ['puzzle', 'logic puzzle', 'brain teaser', 'daily puzzle', 'critter logic',
'puzzle game'],
    },
    'fr': {
        'hook': 'Sauras-tu|le résoudre ?', 'pause': 'Mets pause et essaie', 'solved': 'Résolu
!',
        'nests': 'Nids', 'pairs': 'Paires', 'trail': 'Sentier',
        'nests_rules': 'Une bestiole par ligne, colonne et zone. Elles ne se touchent jamais !',
        'pairs_rules': 'Autant de soleils que de lunes par ligne, jamais 3 pareils de suite.',
        'trail_rules': 'Un seul sentier par toutes les cases, les pierres dans l’ordre.',
        'free': 'Gratuit sur Google Play', 'soon': 'Bientôt sur Google Play', 'end_sub': 'De
nouveaux défis chaque jour',
        'title': 'Sauras-tu résoudre ce casse-tête {game} ? Défi n° {n} #shorts',
        'desc': ('Défi n° {n} : {rules}\nMets pause et essaie avant la solution !\n\n'
                 'Critter Logic : 3 nouveaux casse-têtes logiques chaque jour (Nids, Paires,
Sentier), des bestioles '
                 'à collectionner. {free} :\n{url}\n\n#cassetete #logique #defi #critterlogic'),
        'tags': ['casse-tête', 'logique', 'jeu de logique', 'défi du jour', 'critter logic'],
    },
    'es': {
        'hook': '¿Puedes|resolverlo?', 'pause': 'Pausa e inténtalo', 'solved': '¡Resuelto!',
        'nests': 'Nidos', 'pairs': 'Parejas', 'trail': 'Sendero',
        'nests_rules': 'Un animalito por fila, columna y zona. ¡Nunca se tocan!',
        'pairs_rules': 'Tantos soles como lunas en cada línea, nunca 3 iguales seguidos.',
        'trail_rules': 'Un solo sendero por todas las casillas, las piedras en orden.',
        'free': 'Gratis en Google Play', 'soon': 'Muy pronto en Google Play', 'end_sub': 'Retos
nuevos cada día',
        'title': '¿Puedes resolver este reto {game}? Reto n.º {n} #shorts',
        'desc': ('Reto n.º {n}: {rules}\n¡Pausa e inténtalo antes de la solución!\n\n'
                 'Critter Logic: 3 retos de lógica nuevos cada día (Nidos, Parejas, Sendero),
animalitos para '
                 'coleccionar. {free}:\n{url}\n\n#rompecabezas #logica #retodiario
#critterlogic'),
```

```
        'tags': ['rompecabezas', 'lógica', 'reto diario', 'juego de lógica', 'critter logic'],
    },
}

# ------------------------------------------------------------------------------------------
# Which puzzle


def epoch():
    """Daily #1's date, read from lib/daily.dart so the numbers always match the app."""
    src = open(os.path.join(ROOT, 'lib', 'daily.dart')).read()
    y, m, d = map(int, re.search(r'dailyEpoch = DateTime\.utc\((\d+), (\d+), (\d+)\)',
src).groups())
    return dt.date(y, m, d)


def pick(date):
    """(game, daily number) for the Short of [date]: yesterday's daily, games in turn."""
    n = (date - epoch()).days  # daily number of the day before
    if n < 1:
        raise SystemExit(f'{date}: no daily before it yet (daily #1 is {epoch()})')
    return GAMES[date.toordinal() % 3], n


# ------------------------------------------------------------------------------------------
# Recording


def record(game, n, lang, path):
    """Plays daily [n]'s [game] from its empty board; returns (first move, result screen) in
seconds."""
    p = daily(n)[game]
    launch(lang, reset=True, daily=n, boards='false')
    taps([PLAY[game]])
    time.sleep(1.6)
    rec_start(path)
    time.sleep(2.0)
    if game == 'nests':
        g = grid(NESTS_BOX, p['s'])
        for c in nests_cells(p):
            taps([g(c), g(c)], 0.12)
            time.sleep(0.75)
    elif game == 'pairs':
        for c in [i for i in range(p['s'] * p['s']) if p['g'][i] == '.']:
            taps(pairs_taps(p, c, PAIRS_BOX), 0.06)
            time.sleep(0.12)
    else:
        drag(trail_drag_points(p), hold=0.3, step=0.07)
    time.sleep(6.5)
    rec_stop()
    d = diffs(path)
    t = lambda i: round((i + 1) / 10, 2)  # noqa: E731
    first = next(t(i) for i, v in enumerate(d) if v >= 0.4 and t(i) > 1.2)
    result = next((t(i) for i, v in enumerate(d) if v >= 10 and t(i) > first + 1.0), None)
    if result is None:
        raise SystemExit(f'{path}: no result screen found (unsolved?)')
    return first, result


# ------------------------------------------------------------------------------------------
# Drawing


def font(size, semi=False):
    return ImageFont.truetype(FONT_SEMI if semi else FONT, size)
```

```
def wrap(draw, text, f, width):
    lines, line = [], ''
    for word in text.split():
        test = (line + ' ' + word).strip()
        if draw.textlength(test, font=f) <= width or not line:
            line = test
        else:
            lines.append(line)
            line = word
    return lines + [line]


def outlined(draw, xy, text, f, fill, stroke=10, anchor='mm'):
    draw.text(xy, text, font=f, fill=fill, anchor=anchor, stroke_width=stroke, stroke_fill=INK)


def rounded(img, radius):
    mask = Image.new('L', (img.width * 2, img.height * 2), 0)
    ImageDraw.Draw(mask).rounded_rectangle((0, 0, mask.width - 1, mask.height - 1), radius * 2,
fill=255)
    out = img.convert('RGBA')
    out.putalpha(mask.resize(img.size, Image.LANCZOS))
    return out


def backdrop(still):
    """The canvas behind the phone frame: the first frame blown up, blurred and darkened."""
    bg = still.resize((W, int(still.height * W / still.width))).crop((0, 0, W, H)) if
still.height * W / still.width >= H \
        else still.resize((int(still.width * H / still.height), H)).crop((0, 0, W, H))
    bg = bg.filter(ImageFilter.GaussianBlur(40))
    return Image.blend(bg, Image.new('RGB', (W, H), INK), 0.45)


def pop(t, start, dur=0.35):
    """0 → 1 with an overshoot, for captions popping in."""
    x = min(max((t - start) / dur, 0.0), 1.0)
    if x >= 1:
        return 1.0
    s = 1.9
    x -= 1
    return 1 + (s + 1) * x ** 3 + s * x ** 2


def scaled(img, k, center):
    if k <= 0.01:
        return None, None
    im = img.resize((max(1, int(img.width * k)), max(1, int(img.height * k))), Image.LANCZOS)
    return im, (int(center[0] - im.width / 2), int(center[1] - im.height / 2))


def caption(lines, size, color=GOLD, width=W - 120):
    """A transparent caption image; lines = list of strings (all drawn in [color])."""
    f = font(size)
    probe = ImageDraw.Draw(Image.new('RGBA', (1, 1)))
    rows = []
    for l in lines:
        rows += wrap(probe, l, f, width)
    lh = int(size * 1.12)
    img = Image.new('RGBA', (W, lh * len(rows) + 40), (0, 0, 0, 0))
    d = ImageDraw.Draw(img)
    for i, r in enumerate(rows):
        outlined(d, (W // 2, 20 + lh * i + lh // 2), r, f, color, stroke=max(6, size // 9))
    return img


def pause_pill(text):
    f = font(54, semi=True)
    d = ImageDraw.Draw(Image.new('RGBA', (1, 1)))
    tw = int(d.textlength(text, font=f))
```

```
    w, h = tw + 170, 110
    img = Image.new('RGBA', (w, h), (0, 0, 0, 0))
    d = ImageDraw.Draw(img)
    d.rounded_rectangle((0, 0, w - 1, h - 1), h // 2, fill=(255, 255, 255, 240), outline=INK,
width=6)
    for x in (52, 76):  # the pause sign
        d.rounded_rectangle((x, 30, x + 14, h - 30), 5, fill=INK)
    d.text((118, h // 2), text, font=f, fill=INK, anchor='lm')
    return img


def on_play():
    """Whether the game is live on Google Play yet (until then the Shorts say "coming soon")."""
    import urllib.error
    import urllib.request
    try:
        return urllib.request.urlopen(PLAY_URL + '&hl=en', timeout=20).status == 200
    except urllib.error.HTTPError:
        return False


def end_card(lang, live):
    tx = TEXT[lang]
    store = tx['free' if live else 'soon']
    img = Image.new('RGBA', (W, H), (0, 0, 0, 0))
    d = ImageDraw.Draw(img)
    d.rectangle((0, 0, W, H), fill=(*INK, 205))
    icon = Image.open(os.path.join(ROOT, 'store', 'icon-512.png')).convert('RGBA').resize((420,
420), Image.LANCZOS)
    img.alpha_composite(rounded(icon, 90), ((W - 420) // 2, 470))
    outlined(d, (W // 2, 1030), 'Critter Logic', font(124), (255, 255, 255), stroke=12)
    outlined(d, (W // 2, 1170), tx['end_sub'], font(64, semi=True), GOLD, stroke=8)
    f = font(66)
    tw = int(d.textlength(store, font=f))
    d.rounded_rectangle(((W - tw) // 2 - 60, 1300, (W + tw) // 2 + 60, 1430), 65, fill=(52, 168,
83), outline=INK,
                        width=6)
    d.text((W // 2, 1365), store, font=f, fill=(255, 255, 255), anchor='mm')
    return img


# ---------------------------------------------------------------------------------------------
---------------------
# Edit


def frames_of(path, start, length, speed=1.0):
    """The cropped recording at FPS from [start] for [length] seconds, sped up [speed] times, as
PIL images (streamed)."""
    x0, y0, x1, y1 = CROP
    cw, ch = x1 - x0, y1 - y0
    cmd = [FFMPEG, '-v', 'error', '-ss', f'{start:.3f}', '-t', f'{length:.3f}', '-i', path,
           '-vf', f'setpts=PTS/{speed:.4f},fps={FPS},crop={cw}:{ch}:{x0}:{y0}', '-f',
'rawvideo', '-pix_fmt', 'rgb24', '-']
    p = subprocess.Popen(cmd, stdout=subprocess.PIPE)
    size = cw * ch * 3
    try:
        while True:
            raw = p.stdout.read(size)
            if len(raw) < size:
                break
            yield Image.frombytes('RGB', (cw, ch), raw)
    finally:
        p.kill()
        p.wait()


def soundtrack(path, length, solve_at):
    import make_promo_music as music
    rate = music.RATE
```

```
    st = music.render()
    st = st / np.max(np.abs(st)) * 0.55
    start = int(4.0 * rate)  # skip the trailer's cold open: straight into the groove
    n = int(length * rate)
    mix = st[start:start + n].copy()
    if len(mix) < n:
        mix = np.concatenate([mix, np.zeros((n - len(mix), 2))])
    fade = int(1.2 * rate)
    mix[-fade:] *= np.linspace(1, 0, fade)[:, None]
    mix[:int(0.05 * rate)] *= np.linspace(0, 1, int(0.05 * rate))[:, None]
    with wave.open(os.path.join(ROOT, 'assets', 'sfx', 'solve.wav')) as w:
        sfx = np.frombuffer(w.readframes(w.getnframes()), '<i2').astype(np.float32) / 32768
        if w.getnchannels() == 2:
            sfx = sfx.reshape(-1, 2)
        else:
            sfx = np.repeat(sfx[:, None], 2, axis=1)
        if w.getframerate() != rate:
            idx = np.arange(0, len(sfx), w.getframerate() / rate).astype(int)
            sfx = sfx[idx[idx < len(sfx)]]
    at = int(solve_at * rate)
    end = min(n, at + len(sfx))
    mix[at:end] += sfx[:end - at] * 0.9
    music.write_wav(path, mix)


def edit(clip, first, result, game, n, lang, out, live):
    tx = TEXT[lang]
    # The app's result screen is cut (with boards=false it says the leaderboard is unavailable):
the Short holds the
    # solved board instead.
    solve_start = max(0.0, first - 0.25)
    # Long solves (big Pairs boards) are sped up to about 9 s.
    speed = max(1.0, (result - 0.15 - solve_start) / MAX_PLAY)
    frames = frames_of(clip, solve_start, result - 0.15 - solve_start, speed)
    play_len = (result - 0.15 - solve_start) / speed
    solve_len = play_len + HOLD
    total = HOOK + solve_len + END
    first_frame = next(frames)
    bg = backdrop(first_frame)
    # Everything that must be read stays above y ~1550: the Shorts player covers the bottom with
the title and channel.
    phone_w = 900
    phone_h = int(first_frame.height * phone_w / first_frame.width)
    py = 480
    hook = caption(tx['hook'].split('|'), 120)
    rules = caption([tx[game + '_rules']], 54, (255, 255, 255))
    label = caption([f"{tx[game]} · #{n}"], 64)
    pill = pause_pill(tx['pause'])
    solved = caption([tx['solved']], 130)
    card = end_card(lang, live)

    wav = out[:-4] + '.wav'
    soundtrack(wav, total, HOOK + play_len - 0.1)
    enc = subprocess.Popen([FFMPEG, '-v', 'error', '-y', '-f', 'rawvideo', '-pix_fmt', 'rgb24',
'-s', f'{W}x{H}',
                            '-r', str(FPS), '-i', '-', '-i', wav, '-c:v', 'libx264', '-preset',
'medium', '-crf', '20',
                            '-pix_fmt', 'yuv420p', '-c:a', 'aac', '-b:a', '160k',
                            '-af', 'loudnorm=I=-14:TP=-1.5:LRA=11', '-shortest', '-movflags',
'+faststart', out],
                           stdin=subprocess.PIPE)
    last = first_frame
    for i in range(int(round(total * FPS))):
        t = i / FPS
        if HOOK <= t < HOOK + play_len:
            last = next(frames, last)
        canvas = bg.copy().convert('RGBA')
        phone = rounded(last.resize((phone_w, phone_h), Image.BILINEAR), 48)
        # A slow push-in on the empty board, then the phone settles back for the solve.
        k = 1.0 + 0.04 * min(t / HOOK, 1.0) if t < HOOK else 1.04 - 0.04 * min((t - HOOK) / 0.4,
```

```
1.0)
        im, at = scaled(phone, k, (W // 2, py + phone_h // 2))
        canvas.alpha_composite(im, at)
        if t < HOOK:
            im, at = scaled(hook, pop(t, 0.1), (W // 2, 150))
            if im:
                canvas.alpha_composite(im, at)
            im, at = scaled(rules, pop(t, 0.5), (W // 2, 380))
            if im:
                canvas.alpha_composite(im, at)
            im, at = scaled(pill, pop(t, 1.0), (W // 2, py + phone_h - 110))
            if im:
                canvas.alpha_composite(im, at)
        elif t < HOOK + solve_len:
            done = HOOK + play_len
            if t < done:
                canvas.alpha_composite(label, (0, 230 - label.height // 2))
            im, at = scaled(solved, pop(t, done), (W // 2, 240))
            if im:
                canvas.alpha_composite(im, at)
        else:
            a = min((t - HOOK - solve_len) / 0.3, 1.0)
            c = card.copy()
            c.putalpha(c.getchannel('A').point(lambda v: int(v * a)))
            canvas.alpha_composite(c)
        enc.stdin.write(canvas.convert('RGB').tobytes())
    enc.stdin.close()
    enc.wait()
    os.remove(wav)
    return total


def render(dates, lang, rerecord=False, live=None):
    os.makedirs(OUT, exist_ok=True)
    live = on_play() if live is None else live
    if not live:
        print('not on Google Play yet: the Shorts say "coming soon"', flush=True)
    old = setup()
    try:
        for date in dates:
            game, n = pick(date)
            tx = TEXT[lang]
            clip = os.path.join(OUT, f'{date}-{lang}-raw.mp4')
            events = clip[:-4] + '.json'
            print(f'{date}: {game} daily #{n} ({lang})', flush=True)
            if rerecord or not os.path.exists(events):
                first, result = record(game, n, lang, clip)
                json.dump({'first': first, 'result': result}, open(events, 'w'))
            else:  # re-edit the earlier recording
                ev = json.load(open(events))
                first, result = ev['first'], ev['result']
            out = os.path.join(OUT, f'{date}-{lang}.mp4')
            length = edit(clip, first, result, game, n, lang, out, live)
            meta = {'date': str(date), 'lang': lang, 'game': game, 'daily': n, 'length':
round(length, 1),
                    'title': tx['title'].format(game=tx[game], n=n),
                    'description': tx['desc'].format(n=n, rules=tx[game + '_rules'],
url=PLAY_URL,
                                                     free=tx['free' if live else 'soon']),
                    'tags': tx['tags']}
            json.dump(meta, open(out[:-4] + '.json', 'w'), indent=1, ensure_ascii=False)
            print(f'  {out} ({length:.1f} s)', flush=True)
    finally:
        teardown(old)


# ---------------------------------------------------------------------------------------------
---------------------
# Upload
```

```
def youtube():
    from google.auth.transport.requests import Request
    from google.oauth2.credentials import Credentials
    from google_auth_oauthlib.flow import InstalledAppFlow
    from googleapiclient.discovery import build
    scopes = ['https://www.googleapis.com/auth/youtube.upload']
    creds = Credentials.from_authorized_user_file(TOKEN, scopes) if os.path.exists(TOKEN) else
None
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    if not creds or not creds.valid:
        if not os.path.exists(CLIENT_SECRET):
            raise SystemExit(f'missing {CLIENT_SECRET}: see store/shorts/README.md')
        creds = InstalledAppFlow.from_client_secrets_file(CLIENT_SECRET,
scopes).run_local_server(port=0)
    with open(TOKEN, 'w') as f:
        f.write(creds.to_json())
    os.chmod(TOKEN, 0o600)
    return build('youtube', 'v3', credentials=creds)


def upload(dry_run):
    from googleapiclient.errors import HttpError
    from googleapiclient.http import MediaFileUpload
    ledger = json.load(open(LEDGER)) if os.path.exists(LEDGER) else {}
    todo = []
    for name in sorted(os.listdir(OUT)) if os.path.isdir(OUT) else []:
        if name.endswith('.json') and not name.endswith('-raw.json'):
            meta = json.load(open(os.path.join(OUT, name)))
            key = f"{meta['date']}-{meta['lang']}"
            when = dt.datetime.combine(dt.date.fromisoformat(meta['date']),
dt.time(PUBLISH_HOUR),
                                       ZoneInfo(PUBLISH_TZ)).astimezone(dt.timezone.utc)
            if key not in ledger and when > dt.datetime.now(dt.timezone.utc):
                todo.append((key, meta, when))
    if not todo:
        print('nothing to upload')
        return
    yt = None if dry_run else youtube()
    for key, meta, when in todo:
        body = {
            'snippet': {'title': meta['title'][:100], 'description': meta['description'],
'tags': meta['tags'],
                        'categoryId': '20', 'defaultLanguage': meta['lang'],
'defaultAudioLanguage': 'zxx'},
            'status': {'privacyStatus': 'private', 'publishAt':
when.isoformat().replace('+00:00', 'Z'),
                       'selfDeclaredMadeForKids': False, 'containsSyntheticMedia': False},
        }
        print(f"{key}: {meta['title']} -> public at {when:%Y-%m-%d %H:%M} UTC", flush=True)
        if dry_run:
            continue
        media = MediaFileUpload(os.path.join(OUT, key + '.mp4'), mimetype='video/mp4',
resumable=True)
        req = yt.videos().insert(part='snippet,status', body=body, media_body=media)
        res = None
        try:
            while res is None:
                _, res = req.next_chunk()
        except HttpError as e:
            if 'quotaExceeded' in str(e) or 'uploadLimitExceeded' in str(e):
                print('  YouTube upload quota used up for today: run upload again tomorrow',
flush=True)
                return
            raise
        ledger[key] = {'id': res['id'], 'publishAt': body['status']['publishAt'], 'title':
meta['title']}
        os.makedirs(os.path.dirname(LEDGER), exist_ok=True)
        json.dump(ledger, open(LEDGER, 'w'), indent=1, ensure_ascii=False)
```

```
        print(f"  https://youtube.com/shorts/{res['id']}", flush=True)


def main():
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest='cmd', required=True)
    r = sub.add_parser('render')
    r.add_argument('date', nargs='?', help='first date (default: tomorrow)')
    r.add_argument('--days', type=int, default=1)
    r.add_argument('--lang', default='en', choices=sorted(TEXT))
    r.add_argument('--live', action='store_true', default=None,
                   help='say "Free on Google Play" even if the store page is not up yet (launch
day)')
    r.add_argument('--rerecord', action='store_true', help='record again even if the day was
recorded before')
    sub.add_parser('login', help='sign in to the YouTube channel once (saves the token)')
    u = sub.add_parser('upload')
    u.add_argument('--dry-run', action='store_true')
    a = ap.parse_args()
    if a.cmd == 'render':
        first = dt.date.fromisoformat(a.date) if a.date else dt.date.today() +
dt.timedelta(days=1)
        render([first + dt.timedelta(days=i) for i in range(a.days)], a.lang, a.rerecord,
a.live)
    elif a.cmd == 'login':
        youtube()
        print('signed in, token saved to', TOKEN)
    else:
        upload(a.dry_run)


if __name__ == '__main__':
    main()
```